Foundations of Deep Learning Pipelines and Architectures From Linear Models to Modern Sequence Models: A Comparative Study for Image Classification

Group 99 – PTQ

Nguyen Minh Phuc Nguyen Thanh The Nguyen Minh Quang

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology
Vietnam National University Ho Chi Minh City

Course: Deep Learning and Its Applications (CO3133) – Semester 261
Instructor: Dr. Le Thanh Sach

October 2026

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Table of Contents

1. Problem and Data Description
2. Methodology and Model Architectures
3. Experimental Setup and Protocol
4. Implementation Results
5. Limitations, Reproducibility and Conclusion



Table of Contents

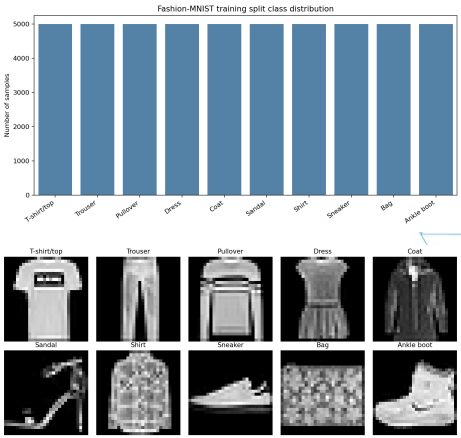
1. Problem and Data Description
2. Methodology and Model Architectures
3. Experimental Setup and Protocol
4. Implementation Results
5. Limitations, Reproducibility and Conclusion

Problem Formulation

- **Task:** Single-label multiclass image classification ($C = 10$). One prediction per full image.
- **Input:** A grayscale product image $x \in \mathbb{R}^{1\times 28\times 28}$.
- **Output:** A neural network f_θ outputs raw, unnormalized logits $z = f_\theta(x) \in \mathbb{R}^{10}$.
- **Decision Rule:** The predicted class is $\hat{y}(x) = \arg\max_k z_k$.

Dataset: Fashion-MNIST

- **Source:** Zalando Research. Used for all reported benchmark metrics (standard MNIST was only used for initial pipeline debugging).
- **Volume:** 70,000 total images.
- **Classes (10):** `T-shirt/top`, `Trouser`, `Pullover`, `Dress`, `Coat`, `Sandal`, `Shirt`, `Sneaker`, `Bag`, `Ankle boot`.
- **Distribution:** Perfectly balanced exactly 10% per class across all splits.
- **Pixel Statistics:** Scaled to $[0, 1]$ range. Global mean $\mu \approx 0.286$, standard deviation $\sigma \approx 0.353$.



Data Splitting & Leakage Prevention

Shared Stratified Partition (`split_seed = 42`)

Split Subset	Total Samples	Per Class	Primary Purpose
Training	$50,000$	$5,000$	Parameter fitting via Backpropagation
Validation	$10,000$	$1,000$	Early stopping & checkpoint selection (`best.pt`)
Official Test	$10,000$	$1,000$	Held-out final evaluation (accessed only once)

- **Seed Isolation:** The split generation uses a permanent `seed=42`. Model training seeds (`42`, `123`, `2026`) only affect weight initialization and batch shuffling, preventing data regeneration.
- **Automated Verification:** Runtime assertions guarantee zero data leakage ($\text{Train} \cap \text{Val} = \emptyset$) and verify exact class counts before any training begins.

Preprocessing & Pipeline Contract

Preprocessing Policy

- **Intensity Scaling:** Raw `uint8` pixels converted to `float32` in the range $[0.0, 1.0]$ via `ToTensor()`.
- **Resolution:** Kept at native 28×28.
- **No Augmentation:** Random crops, flips, and Z-score normalization are strictly disabled to ensure all architectures receive the exact same baseline tensor.

DataLoader Setup

- **Batch Size:** $B = 128$ for all sets.
- **Shuffling:** Enabled for Training; deterministic sequential order for Validation and Test.

Unified Tensor Interface

Component	Shape	Type
Input Images	`[128, 1, 28, 28]`	`float32`
Class Targets	`[128]`	`int64`

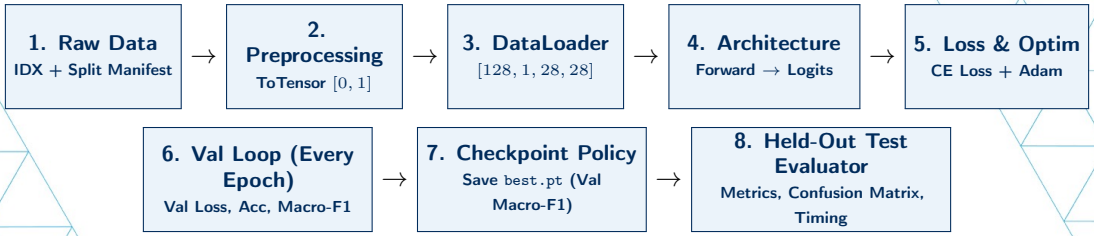
Table of Contents

1. Problem and Data Description
2. Methodology and Model Architectures
3. Experimental Setup and Protocol
4. Implementation Results
5. Limitations, Reproducibility and Conclusion



Shared Pipeline & Implementation Scope

End-to-End Training & Evaluation Pipeline (Shared Across All 5 Models)



Implementation Breakdown

- **Group-Implemented:** Model architectures, tensor reshaping logic, seeded DataLoader, custom train/val loops, early stopping, and CUDA timing wrappers.
- **External Primitives Used:** `torch.nn` (Linear, Conv2d, GRU, Transformer, CE Loss, Adam), `torchvision` (FashionMNIST), `scikit-learn` (metrics, stratified split).

Output Contract & Loss Function

Multiclass Cross-Entropy on Raw Logits

- **Output Contract:** Models map input batch $X \in \mathbb{R}^{B\times 1\times 28\times 28}$ directly to raw logits $Z \in \mathbb{R}^{B\times 10}$.
- **Per-Sample Loss:** For ground-truth y and logit vector z, PyTorch's `nn.CrossEntropyLoss` computes:

$$\ell(z, y) = -z_y + \log\left(\sum_{k=0}^{9} \exp(z_k)\right)$$

- **Epoch Loss:** Sample-weighted average across batches:

$$\mathcal{L}_{\text{epoch}} = \frac{1}{\sum_b N_b} \sum_b N_b \cdot \bar{\ell}_b$$

Why No Softmax Before CrossEntropyLoss?

1. **Numerical Stability:** PyTorch internally fuses `LogSoftmax` and `NLLLoss`, applying the *Log-Sum-Exp trick* with $M = \max_k z_k$:

$$\log \sum_{k=0}^{9} e^{z_k} = M + \log \sum_{k=0}^{9} e^{z_k - M}$$

   This prevents floating-point overflow and underflow.

2. **Mathematical Correctness:** Applying `Softmax` prior to `CrossEntropyLoss` evaluates a double exponential (log of a softmax of probabilities), destroying gradient flow and stalling convergence.

Overview: How the 5 Models Read an Image

Input Representation Transformation from Shared Batch `[B, 1, 28, 28]`

Model Family	Input Transformation	Internal Tensor View	Structural Unit	Inductive Bias
1. **Linear**	`Flatten(start_dim=1)`	`[B, 784]`	784 independent pixels	Linear hyperplanes in $\mathbb{R}^{784}$
2. **MLP**	`Flatten(start_dim=1)`	`[B, 784]`	784 unordered features	Global dense non-linear mixing
3. **CNN**	Preserve 2D Grid	`[B, 1, 28, 28]`	3×3 local neighborhoods	Spatial locality & shift invariance
4. **GRU** (`rnn`)	`squeeze(1)` (Row-wise)	`[B, 28, 28]`	$T = 28$ rows of 28 pixels	Sequential top-to-bottom memory
5. **Transformer**	`squeeze(1)` + Linear Proj	`[B, 28, 64]`	$T = 28$ row tokens ($d = 64$)	All-to-all global row attention

Flattened vs. Spatial Bias

- **Linear/MLP:** Discards 2D coordinates; invariant to fixed pixel permutations.
- **CNN:** Preserves local 2D neighborhoods to extract garment edges and contours.

Row-Sequence Paradigm

- **GRU & Transformer:** Treat the image as a sequence of 28 horizontal scanlines.
- Enables a controlled comparison of recurrence vs. self-attention on identical tokens.

Model 1: Linear Classifier

Tensor Transformation Flow: `[B, 1, 28, 28]` $\xrightarrow{\texttt{Flatten}}$ `[B, 784]` $\xrightarrow{\texttt{Linear(784, 10)}}$ `[B, 10]`

Mathematical Formulation

- Flattens each image $x \in \mathbb{R}^{1\times 28\times 28}$ into $v \in \mathbb{R}^{784}$.
- Computes 10 class logits via a single affine mapping:

$$z = Wv + b, \quad W \in \mathbb{R}^{10\times 784},\ b \in \mathbb{R}^{10}$$

- Prediction: $\hat{y} = \arg\max_k z_k$ (raw logits passed directly to `CrossEntropyLoss`).

Exact Parameter Count

$$P_{\text{Linear}} = 784 \times 10 + 10 = \boxed{\mathbf{7{,}850}}$$

Layer-by-Layer Breakdown

Layer	Output	Weights / Bias	Params
Flatten	$[B, 784]$	–	0
Linear (W)	$[B, 10]$	10×784	7,840
Linear (b)	$[B, 10]$	10	10
Total	$[B, 10]$		**7,850**

Role & Structural Limits

- Fast, ultra-compact linear separability baseline.
- Forms flat hyperplane boundaries; ignores 2D spatial neighborhoods and nonlinear interactions.

Model 2: Multilayer Perceptron (MLP) – Architecture

Feed-Forward Formulation ($784 \to 256 \to 128 \to 10$)

- Starts from flattened vector $v \in \mathbb{R}^{784}$ and inserts two dense hidden layers:

$$h_1 = \mathrm{ReLU}(W_1 v + b_1), \qquad \tilde{h}_1 = \mathrm{Dropout}_{0.2}(h_1)$$
$$h_2 = \mathrm{ReLU}(W_2 \tilde{h}_1 + b_2), \qquad \tilde{h}_2 = \mathrm{Dropout}_{0.2}(h_2)$$
$$z = W_3 \tilde{h}_2 + b_3 \in \mathbb{R}^{10}$$

- **Learned Projection:** $784 \to 256$ forms new weighted combinations of *all* 784 pixels (not pixel deletion).

Activation & Regularization Roles

- **Why ReLU is Essential:** Without $\mathrm{ReLU}(a) = \max(0, a)$, stacked affine layers collapse into a single linear transformation ($W_3 W_2 W_1 v + b'$), offering zero extra expressiveness over Model 1.
- **Dropout ($p = 0.2$):** Randomly zeros 20% of hidden activations during training to reduce co-adaptation; disabled during evaluation.
- **Limitation:** Still discards 2D spatial topology upon initial flattening.

Model 2: MLP – Shapes & Parameter Breakdown

Forward Shape Progression & Trainable Parameters

Stage	Operation	Output Shape	Parameters
Input	Flatten	$[B, 784]$	0
Hidden 1	Linear $784 \to 256$, ReLU, Dropout 0.2	$[B, 256]$	200,960
Hidden 2	Linear $256 \to 128$, ReLU, Dropout 0.2	$[B, 128]$	32,896
Classifier	Linear $128 \to 10$	$[B, 10]$	1,290
	Total Trainable Parameters (P_{MLP})		**235,146**

Closed-Form Parameter Math

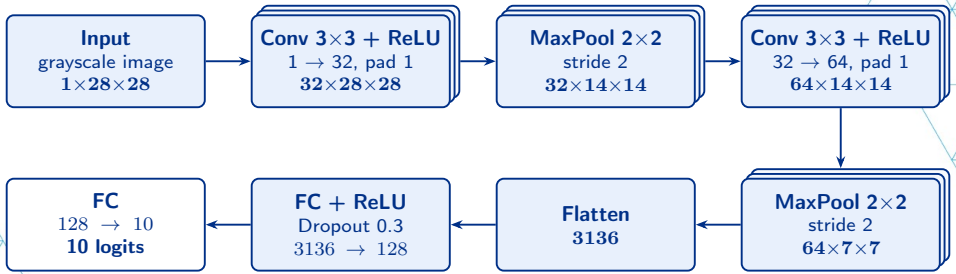
- Each dense layer uses $(n_{\text{in}} + 1)n_{\text{out}}$ parameters:

$$P_{\text{MLP}} = 785 \times 256 + 257 \times 128 + 129 \times 10 = \boxed{235{,}146}$$

Capacity Distribution

- **85.5%** of parameters $(200{,}960)$ sit in Hidden 1 due to dense all-to-all pixel connections.
- ReLU and Dropout add 0 parameters.

Model 3: Convolutional Neural Network (CNN) – Architecture



- **2D Cross-Correlation:** Reuses local filters $K_{j,c}$ across all spatial locations: $Y_j = \sum_{c=1}^{C_{\text{in}}} X_c \star K_{j,c} + b_j$.
- **Spatial Sizing:** $H_{\text{out}} = \lfloor \frac{H_{\text{in}}+2p-k}{s} \rfloor + 1$.
- Conv (k=3, p=1, s=1) preserves size; MaxPool (2×2, s=2) halves resolution ($28 \to 14 \to 7$).

Model 3: CNN – Parameter Math & Trade-offs

Layer-by-Layer Parameter Breakdown

Layer	Output	Calculation	Params
Conv1 ($1 \to 32$)	$32 \times 28 \times 28$	$(3 \cdot 3 \cdot 1 + 1) \times 32$	320
Conv2 ($32 \to 64$)	$64 \times 14 \times 14$	$(3 \cdot 3 \cdot 32 + 1) \times 64$	18,496
FC1 ($3136 \to 128$)	128	$(3136 + 1) \times 128$	401,536
FC2 ($128 \to 10$)	10	$(128 + 1) \times 10$	1,290
Total (P_{CNN})			**421,642**

$$P_{\text{CNN}} = 320 + 18{,}496 + 401{,}536 + 1{,}290 = \boxed{421{,}642}$$

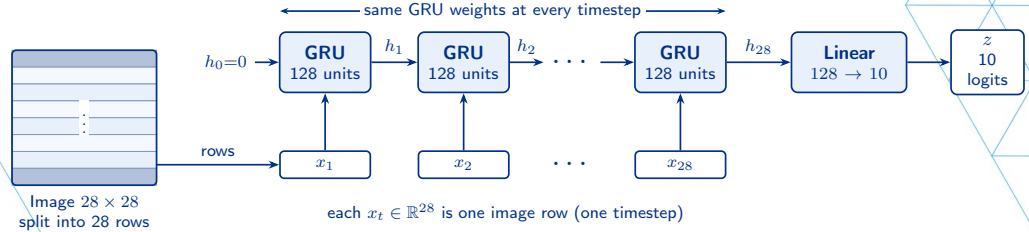
Where Do the Parameters Live?

- **Spatial Filters (4.5%):** Both Conv layers combined require only $18{,}816$ parameters due to weight sharing.
- **Dense Head (95.5%):** `FC1` alone accounts for **95.2%** $(401{,}536)$ of total model size after flattening $64 \times 7 \times 7 = 3{,}136$ features.

Inductive Bias Limits

- Local connectivity directly matches garment edges and silhouettes.
- 2×2 pooling can discard fine collar/sleeve details at low resolution.

Model 4: Gated Recurrent Unit (GRU) – Row Sequence



Gated Recurrence ($t = 1, \ldots, 28$)

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r), \quad u_t = \sigma(W_u x_t + U_u h_{t-1} + b_u)$$

$$\widetilde{h}_t = \tanh\big(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h\big)$$

$$h_t = u_t \odot h_{t-1} + (1 - u_t) \odot \widetilde{h}_t \in \mathbb{R}^{128}$$

State Accumulation Readout

- Processes 28 horizontal scanlines top-to-bottom ($x_t \in \mathbb{R}^{28}$).
- Final state $h_{28} \in \mathbb{R}^{128}$ summarizes the full garment and feeds $z = W h_{28} + b$.

Model 4: GRU – Parameter Math & Trade-offs

Exact Parameter Breakdown ($I = 28, H = 128$)

- PyTorch's `nn.GRU` allocates input-to-hidden weights, hidden-to-hidden weights, and *two* separate bias vectors (b_{ih}, b_{hh}) for each of the 3 gates ($r_t, u_t, \widetilde{h}_t$):

$$\begin{aligned} P_{\text{GRU}} &= 3 \times \big(128 \cdot 28 + 128 \cdot 128 + 2 \cdot 128\big) + (128 + 1) \cdot 10 \\ &= 3 \times \big(3{,}584 + 16{,}384 + 256\big) + 1{,}290 \\ &= 60{,}672 + 1{,}290 = \boxed{\mathbf{61{,}962}} \end{aligned}$$

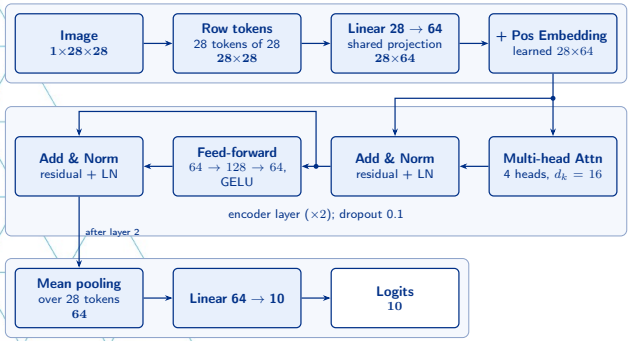
Why Is GRU So Compact?

- Reuses the exact same $60{,}672$ recurrent parameters across all 28 timesteps (not 28 separate layers).
- Avoids the massive dense flattening layer of MLP ($3.8\times$ smaller) and CNN ($6.8\times$ smaller).

Structural Limitations

- Imposes an artificial 1D top-to-bottom order on a static 2D image.
- Sequential step dependency (h_t awaits h_{t-1}) limits parallelization across rows.

Model 5: Transformer Encoder – Architecture



Row Embedding Position

- Projects 28 row tokens ($X \in \mathbb{R}^{28\times 28}$) to $d = 64$ and adds learned position $P \in \mathbb{R}^{28\times 64}$:

$$H_0 = \mathrm{Linear}_{28\to 64}(X) + P$$

- No patch splitting or `[CLS]` token used.

Multi-Head Self-Attention ($h = 4, d_k = 16$)

$$A = \mathrm{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{16}}\right) \in \mathbb{R}^{28\times 28}$$

- Every row compares its query with all 28 keys in parallel.
- Followed by token-wise FFN ($64 \to 128 \to 64$, GELU, dropout 0.1).

Model 5: Transformer Math & 5-Model Summary

Mean Pooling Classifier

$$h_{\text{img}} = \frac{1}{28}\sum_{t=1}^{28} H_{2,t} \in \mathbb{R}^{64}, \quad z = W h_{\text{img}} + b$$

Transformer Parameter Math

$$P_{\text{Tr}} = \underbrace{(28+1)\cdot 64}_{\text{input proj } (1{,}856)} + \underbrace{28 \cdot 64}_{\text{pos } (1{,}792)} + \underbrace{2 \times 33{,}472}_{\text{2 encoder layers } (66{,}944)} + \underbrace{(64+1)\cdot 10}_{\text{head } (650)} = \boxed{\mathbf{71{,}242}}$$

Summary of All 5 Implemented Architectures

Model	Input View	Core Mechanism	Params
Linear	$[B, 784]$	Single affine map	7,850
MLP	$[B, 784]$	Dense $784 \to 256 \to 128$ + ReLU	235,146
CNN	$[B, 1, 28, 28]$	$2\times$Conv3$\times$3 + Pool + FC	421,642
GRU	$[B, 28, 28]$	1-layer GRU (H=128), h_{28}	61,962
Transformer	$[B, 28, 28]$	$2\times$Enc (d=64, h=4) + MeanPool	71,242

- **Trade-off:** Row-token Transformer enables direct all-to-all row interaction with small size (71.2K), but summarizes whole horizontal slices rather than local 2D patches.

Summary Comparison: The Five Architectures

Architectural & Parameter Summary Across All 5 Implemented Models

Model	Input View	Core Building Blocks	Regularization	Primary Inductive Bias	Parameters
Linear	$[B, 784]$	`Linear(784->10)`	Weight decay 10^{-4}	Linear pixel hyperplanes	**7,850**
MLP	$[B, 784]$	`784 -> 256 -> 128 -> 10` (ReLU)	Dropout 0.2 + WD	Unordered non-linear mixing	**235,146**
CNN	$[B, 1, 28, 28]$	$2\times$(`Conv3x3+ReLU+Pool2x2`) + `FC(128)`	Dropout 0.3 + WD	2D spatial locality & shift inv.	**421,642**
GRU	$[B, 28, 28]$	1-Layer `GRU(28->128)` + h_{28} Head	Weight decay 10^{-4}	Top-to-bottom row recurrence	**61,962**
Transformer	$[B, 28, 28]$	`Proj(64)` + `Pos` + $2\times$`Enc(h=4, ff=128)` + MeanPool	Dropout 0.1 + WD	Global all-to-all row attention	**71,242**

Capacity Spectrum ($53.7\times$ Range)

- Model sizes span from 7,850 (Linear) to 421,642 (CNN).
- Row-based **GRU** (61.9K) and **Transformer** (71.2K) are $> \mathbf{3.3}\times$ smaller than the MLP (235.1K).

Fairness of Architectural Comparison

- Because capacity, representation, and internal computation vary across families, predictive results must be interpreted jointly with **parameter count** and **runtime cost**.

Table of Contents

1. Problem and Data Description
2. Methodology and Model Architectures
3. **Experimental Setup and Protocol**
4. Implementation Results
5. Limitations, Reproducibility and Conclusion

Shared Training Protocol & Metric Definitions

Shared Optimization Settings

- **Loss Function:** `CrossEntropyLoss` (on raw logits).
- **Optimizer:** Adam (lr $= 0.001$, weight decay $= 10^{-4}$).
- **Batch & Epochs:** Batch size 128, max 100 epochs (no scheduler).
- **Robustness:** Each model runs on **3 seeds** (42, 123, 2026). Split seed is strictly fixed at 42.

Predictive Quality Metric

- **Accuracy:** Overall fraction of correct predictions across all evaluated test samples.

3-Tier Checkpoint Selection (`best.pt`)

- Validation loop runs every epoch to evaluate metrics.
- **Rule 1 (Primary):** Save new `best.pt` if Validation Macro-F1 improves by > 0.0001.
- **Rule 2 (Tie-breaker):** Select lower Validation Loss if difference is within ± 0.0001.
- **Rule 3 (Patience):** Early stop after 15 stagnant epochs.
- Test evaluator strictly loads `best.pt` (held-out test set).

Primary Selection Metric: Macro-F1

- Unweighted mean of F1 scores across all 10 classes. Penalizes models that sacrifice difficult upper-body classes for easy ones.

Experimental Design & Runtime Methodology

Controlled Experimental Design

- **Manipulated Factor:** Neural architecture family and its input representation (Flatten vs. Spatial vs. Sequence).
- **Controlled Factors:** Dataset partition, normalization, optimizer settings, early-stopping rule, and training seeds.
- **Goal:** Compare predictive scores and computational costs without attributing differences to a single causal factor (since capacity and regularization also vary).

GPU Runtime Methodology (RTX 4050)

- **Training Time:** Wall-clock time around the epoch loop (includes validation & disk I/O).
- **Inference Time (ms/sample):**
  1. 5 warm-up batches to init CUDA.
  2. 3 full passes over the test set.
  3. Transfer inputs to GPU *before* the timer.
  4. `torch.cuda.synchronize()` around the forward pass.

Table of Contents

1. Problem and Data Description
2. Methodology and Model Architectures
3. Experimental Setup and Protocol
4. **Implementation Results**
5. Limitations, Reproducibility and Conclusion

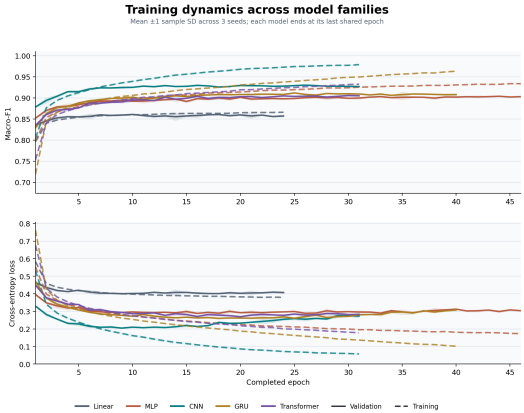
Training Behavior & Convergence

Saved Benchmark Run Inventory

Model	Seeds	Best-Epoch Range
Linear	42, 123, 2026	9 – 35
MLP	42, 123, 2026	31 – 42
CNN	42, 123, 2026	16 – 21
GRU	42, 123, 2026	25 – 35
Transformer	42, 123, 2026	16 – 47

Optimization Speed Insights

- **CNN:** Narrowest range (16–21). Reaches a strong, consistent validation peak quickly.
- **MLP:** Requires more epochs (31–42) to optimize its nonlinear representations.
- **Transformer:** Widest range (16–47), showing high sensitivity to random



- **Generalization Gap:** Training loss (dashed) continues dropping, but validation (solid) plateaus, confirming the necessity of early stopping.

Quantitative Results & Compute Cost

Table: Test Predictive Performance & Cost (3-Seed Mean $\pm$ SD)

Model Family	Test Accuracy (%)	Test Macro-F1 (%)	Parameters	Inference (ms/sample)
1. Linear	$84.31 \pm 0.09\%$	$84.23 \pm 0.06\%$	7,850	0.00530 ± 0.00350
2. MLP	$89.37 \pm 0.11\%$	$89.31 \pm 0.12\%$	235,146	0.00781 ± 0.00833
3. CNN (Best)	$\mathbf{92.22 \pm 0.39\%}$	$\mathbf{92.20 \pm 0.41\%}$	421,642	0.01265 ± 0.01304
4. GRU	$90.29 \pm 0.19\%$	$90.26 \pm 0.21\%$	61,962	0.01880 ± 0.00734
5. Transformer	$89.40 \pm 0.09\%$	$89.37 \pm 0.13\%$	71,242	0.05044 ± 0.03408

Predictive Findings

- **CNN leads decisively:** $+7.97\%$ Macro-F1 over Linear, $+2.89\%$ over MLP, and $+1.94\%$ over GRU.
- **Stability:** Standard deviations remain $\leq 0.41\%$ across all models and seeds.

Parameter Efficiency

- **GRU** (90.26%) beats **MLP** (89.31%) using nearly $4\times$ **fewer parameters**.
- Transformer (89.37%) matches MLP performance with only 71K parameters, though with higher inference latency.

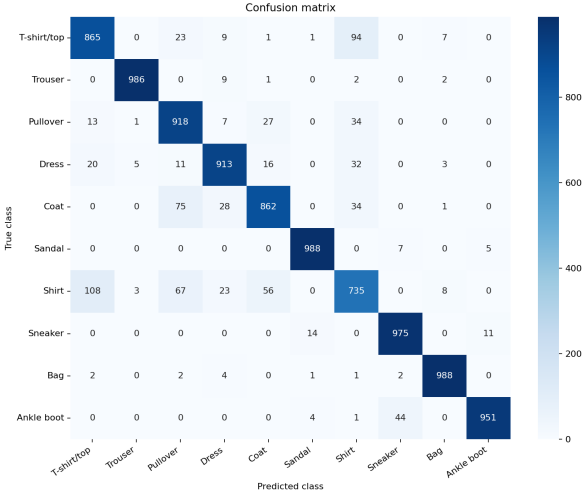
Error Analysis: The Upper-Body Ambiguity (Seed 42)

Most Frequent CNN Confusions

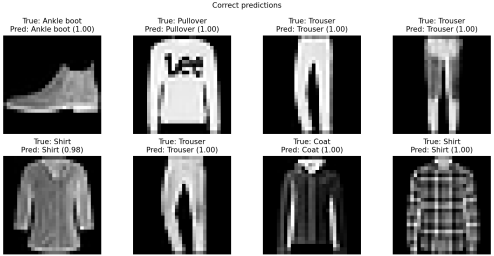
True → Predicted	Errors	Rate
Shirt → T-shirt/top	108	10.8%
T-shirt/top → Shirt	94	9.4%
Coat → Pullover	75	7.5%
Shirt → Pullover	67	6.7%
Shirt → Coat	56	5.6%
Ankle boot → Sneaker	44	4.4%

Key Insights

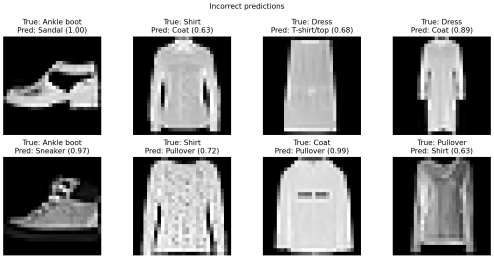
- $\sim 65\%$ **of all errors** are concentrated within just 4 upper-body classes.
- **Shirt** is the weakest class across *all* models (CNN recall: 73.5%, Linear:



Qualitative Assessment: Successes vs. Failures



Correct Predictions: Distinct silhouettes (Trousers, Sneakers) and clear Coat/Shirt features are easily classified.



Incorrect Predictions: High predicted-class confidence (e.g., 1.00 for Sandal) does not guarantee correctness.

- **Root Cause Hypothesis:** At 28×28 grayscale resolution, fine-grained discriminative features (buttons, collar seams, fabric texture) vanish during downsampling, causing upper-body silhouettes to overlap structurally.

Ablation Study: Sequence Representation (Seed 42)

Isolating the Tokenization Strategy

- **Goal:** Check whether the ordering and granularity of the input sequence affect performance, holding the architecture and training protocol fixed.
- **Variants:**
  1. **Rows:** 28 tokens of 28 pixels.
  2. **Columns:** 28 tokens of 28 pixels.
  3. **Patches:** 49 tokens of 4×4 pixels.

Validation Accuracy (Seed 42)

Model	Representation	Best Val Acc
GRU	Rows	**91.19%**
GRU	Columns	90.31%
GRU	4×4 patches	89.67%
Transformer	Rows	**90.63%**
Transformer	Columns	90.29%
Transformer	4×4 patches	89.93%

Ablation Insights

- **Rows > Columns > Patches** for both GRU and Transformer in this setup.
- The recurrent model (GRU) heavily benefits from the natural 28-step horizontal scan. Patch splitting increases sequence length to 49 steps and degrades performance (-1.52%).

Table of Contents

1 Problem and Data Description

2 Methodology and Model Architectures

3 Experimental Setup and Protocol

4 Implementation Results

5 Limitations, Reproducibility and Conclusion

Limitations & Conclusion

Experimental Limitations

- **Dataset Scope:** 28×28 grayscale product photos do not generalize to high-res color retail images with backgrounds.
- **Hyperparameters:** Shared Adam protocol (10^{-3}) ensures fair implementation constraints, but limits each model's theoretical ceiling.
- **Ablation Limits:** Patch tokenization changes both sequence length (49) and token dimension (16), complicating isolation.

Conclusion

- **CNN is the predictive winner** (92.22% Acc, 92.20% Macro-F1), proving that 2D spatial locality is the strongest inductive bias for this task.
- **Parameter Efficiency:** Row-token **GRU** (90.26%) and **Transformer** (89.37%) prove that recurrence and attention can outperform dense **MLP** mixing with a fraction of the capacity.
- Model selection must balance Accuracy vs. Memory (GRU) vs. Latency (Linear/MLP).

Reproducibility & Project Artifacts

Artifact Traceability

- **Repository:** `https://github.com/tapstoptap/deep-learning-261`
- **Environment:** Python 3.11.15, PyTorch 2.5.1+cu121, NVIDIA RTX 4050.
- **Saved Summaries:** `model_comparison.csv` and `run_metrics.csv` in `results_norm_full/`.

Reproduction Commands

```
# 1. Environment
pip install -r requirements.txt

# 2. Train Model (e.g., Linear, seed 42)
python src/train.py --config configs/models/line
  --seed 42 --device cuda --num-workers 0
  --output-dir results_norm_full
  --run-name linear_norm_seed42

# 3. Evaluate held-out test set
python src/evaluate.py --config configs/models/l
  --checkpoint checkpoints/linear/.../best.pt
  --device cuda --allow-test
```



Member Contributions

Group 99 (Choco) – Task Allocation

Member Name	Student ID	Primary Contributions
Nguyen Minh Quang	2453052	Complete implementation of Assignment 1 (EDA, DataLoader preparation; Linear, MLP, CNN, GRU, and Transformer architectures); multi-model benchmarking, performance analysis, error analysis, and final report/slide integration.



References I

1. H. Xiao, K. Rasul, and R. Vollgraf, *Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms*, arXiv:1708.07747, 2017.
2. PyTorch Contributors, *PyTorch: An Open Source Machine Learning Framework*, 2019. `https://pytorch.org/`.
3. F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, 12, 2825–2830, 2011.
4. N. Srivastava et al., *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*, Journal of Machine Learning Research, 15, 1929–1958, 2014.
5. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 86(11), 2278–2324, 1998.
6. K. Cho et al., *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*, Proceedings of EMNLP, 1724–1734, 2014.

References II

7. A. Vaswani et al., *Attention Is All You Need*, Advances in Neural Information Processing Systems 30, 2017.
8. A. Dosovitskiy et al., *An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale*, International Conference on Learning Representations, 2021.
9. D. P. Kingma and J. Ba, *Adam: A Method for Stochastic Optimization*, International Conference on Learning Representations, 2015.
10. L. Prechelt, *Early Stopping—But When?*, In *Neural Networks: Tricks of the Trade*, Springer, 1998.
11. M. Sokolova and G. Lapalme, *A Systematic Analysis of Performance Measures for Classification Tasks*, Information Processing & Management, 45(4), 427–437, 2009.

Thank You For Listening!